

SPEC - the native shell: a native background recorder, native track storage, and the logged rounds moved across

Fable, 2026-09-16, at xhigh on Matt's word. Written for Opus to build at xhigh (his ruling, 2026-09-16: "you can run both on x-high and so can Opus for the build. go ahead"). Fable reviews the diff line by line against this file and signs in docs/DECISIONS_LOG.md.

His words that bound it, verbatim:

- "I really would like this entirely off of webapps and all local."
- "The real motivation is what you stumbled on and it is 2 fold: allowing the GPS to run in the background and mark the track regardless of what I am doing on the phone and 2 eliminating the phone but not with a watch - remember my background."
- "Last thing this should be a native app on my phone"
- Native shell, picked 2026-09-13: "the current screens inside a Kotlin app with their files on the phone, plus a native background recorder. A fully native UI waits for the offseason."
- "I am fixing my hip on October 7th so we have now until then for real data."

Provenance tags: [measured] read from code or data this session, [design] this spec's decision, [verify] Opus confirms during the build.

0. What this is, and what it is not

Is: one Android app, Golf Tracker, that ships the current web screens as its own files, records the GPS track natively for the length of a round no matter what the phone is doing, stores that track natively, and takes his logged rounds across through the export/import path that already exists.

Is not: a rewrite of a screen, a change to the mark pipeline in js/gps/gps.js (field-validated, untouched), a change to the round record's meaning, a change to the export's formatVersion, the course map as app data (a later spec, after the green flow exists), or the watchdog restart from REPORT_2.3.md Section 13 (a follow-up gated on field evidence: *watch, do not change*).

The instrument stays the web build until this passes the bar. v26 on Pages is what he plays this week. The shell feeds a real round only after Section 9's gate, in his chat, on his word.

1. Decisions made at xhigh (each overridable by Matt, in his chat)

#	Decision	Why
D1	The recorder is K's code, lifted from android/bakeoff/app/src/handwritten/ and main/ (fused, high accuracy, 1 Hz, location foreground service, START_STICKY, partial wake lock, resume from a committed active flag). T is not in the shell.	K and T delivered the same fixes byte for byte on the 09-14 round (7,230 of 7,230 shared, n = 1 round, REPORT_2.3.md) [measured]. K holds the only wake lock; the SDK holds none (read from the 4.5.1 binary, 2.3) [measured]. K's code is all in the repo, no licence, no third-party service in the recording path. The recorder sits behind one service contract (Section 3), so T can replace it later behind the same contract if the field says so.
D2	The native fix log is the dense track. trackstore.js routes its public API to the bridge when the shell is present; IndexedDB is not used in the shell.	One source of truth for the track; no catch-up copy, no duplicate points at the boundary. trackstore.js's public functions are already the seam every reader uses (readTrack, writeTrackChunk, trackSize, deleteTrack, trackedRoundIds, pruneOrphanTracks, createTrackWriter) [measured].
D3	Live fixes reach the web layer through a navigator.geolocation shim, exactly the way js/dev/sim.js does it. gps.js is not touched.	sim.js already proves the seam: Object.defineProperty(navigator, 'geolocation', ...) with watchPosition/clearWatch [measured]. The burst reduction, the revive-on-visible logic and every test on GpsService stay as they are.

#	Decision	Why
D4	Rounds stay in the web layer's <code>localStorage</code> , inside the app's private WebView storage. No round-record schema change. <code>schemaVersion</code> does not move. One additive, optional field: <code>round.device.recorder</code> .	Rounds are small; the record's rails (<code>store.js</code> , <code>migrate()</code>) are proven; the export stays <code>formatVersion: 1</code> and every PC tool keeps reading it. App-private WebView storage is deleted only by "clear storage" or uninstall, not by Chrome's origin eviction that took FT1-FT5 (<code>persistence.js</code> header) [measured].
D5	Moving the logged rounds across = export from the PWA, import in the shell. The existing <code>importExport + restoreTracks</code> path, with <code>writeTrackChunk</code> routed to the native store. No new migration code in the round rails.	The path exists and is tested; the acceptance is numeric (Section 8). Nothing changes meaning: an imported track is stored with <code>provider = import</code> so it can never be mistaken for a native recording.
D6	The APK carries no INTERNET permission. The web files are served from the APK's assets through <code>WebViewAssetLoader</code> ; the service worker is not registered in the shell.	" <i>entirely off of webapps and all local</i> " - provable from the manifest. <code>WebViewAssetLoader</code> gives a secure <code>https://appassets.androidplatform.net</code> origin so ES modules load [design; verify]. The SW exists for Pages caching; in the shell it would only cache what is already local.
D7	One native location source. The service is the only <code>FusedLocationProviderClient</code> requester. It runs whenever the web layer holds a watch and the screen is up, or a round is recording. The wake lock and the log are held only while recording.	Two requesters would deliver the same fix twice down two paths. One path means the mark burst and the track are literally the same fixes.
D8	Recording follows the round's status, never a null <code>ctx.round</code> . Start when a round is <code>in_progress</code> ; stop only on completed, abandoned, the round's deletion, or the 8 h cap.	Design rule: no app-driven state change may block logging reality. A WebView that dies mid-round must not take the recording with it (the writer-follows-round pattern in <code>app.js</code> closes on null; the recorder must not).
D9	Version = BUILD.id. The Gradle build reads <code>id: 'vNN'</code> from <code>js/data/build.js</code> and sets <code>versionName</code> to it.	Settings' "Build vNN" and the APK agree by construction; the suite's <code>BUILD.id / gt-shell-<id></code> check is untouched.
D10	REVISION is not bumped here. The shell is a new instrument (<code>js/data/revision.js</code> : <i>which instrument logged this round</i>) and the bump is his call when a build is about to be played.	The wall in <code>CLAUDE.md</code> Section 3. Listed for him in Section 10.

2. The shell (Android project)

- **Location:** `android/tracker/`, its own Gradle project beside `android/bakeoff/` (which stays a frozen test rig - nothing in it changes). Same toolchain as the bake-off [measured]: `compileSdk 36`, `minSdk 30`, `targetSdk 36`, Kotlin, Gradle 8.13, JDK 17, no Node.
- **Application id:** `com.postmaster87.golftracker`. App name Golf Tracker.
- **Web files:** a Gradle task copies `index.html`, `manifest.webmanifest`, `css/`, `js/` (except `js/dev/`), and any icon files `index.html` references from the repo root into the APK's assets at build time. Not copied: `sw.js`, `test/`, `tools/`, `docs/`. One source of truth for the web code; the copy is not committed. [verify] the exact list from `index.html`'s references.
- **Manifest permissions:** `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`, `ACCESS_BACKGROUND_LOCATION`, `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION`, `POST_NOTIFICATIONS`, `WAKE_LOCK`, `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`. **No** `INTERNET`. Portrait.
- **Screens:**
 1. *Setup* (native, shown only while any grant is missing): the bake-off's checklist lifted from `MainActivity` (location services on, precise, allow all the time, notifications, battery unrestricted, and the Samsung never-sleeping line as text). Buttons request each grant. When all are granted the app goes straight to the WebView on every later launch.
 2. *WebView* (the app): `WebViewAssetLoader` serving `/assets/`; JavaScript on; DOM storage on; `WebChromeClient.onShowFileChooser` wired so the Settings import `<input type="file">` opens the system picker (`screen-settings.js:309` [measured]); `addJavascriptInterface(..., "GolfNative")` before the first load. No geolocation permission prompt handling is needed: the web layer never calls the real API in the shell.
- **The bridge object is** `window.GolfNative` (Section 5). Its presence is the one signal the web layer uses to know it is in the shell.

- **versionName** from BUILD.id (D9); versionCode increments per build.

3. The recorder (native)

Lift, do not redesign: RecorderService.kt, Recorder.kt, SessionLog.kt, Sessions.kt, FixCsv.kt, LocationFix.kt, DeviceState.kt, BakeoffApp.logExitsSinceLastLook from the bake-off, renamed into the shell's package. The contract they must keep [measured from the bake-off code]:

- LocationRequest: PRIORITY_HIGH_ACCURACY, interval 1000 ms, min interval 1000 ms, min distance 0 m, max delay 0, GRANULARITY_FINE, setWaitForAccurateLocation(false). Callback on its own HandlerThread.
- Foreground service, type LOCATION, START_STICKY, notification channel IMPORTANCE_LOW, FOREGROUND_SERVICE_IMMEDIATE on 31+.
- Partial wake lock, non-reference-counted, held **only while recording** (D7). Acquired on startRecording, released on stop.
- onTaskRemoved logs task_removed and keeps running. onDestroy logs service_destroy with halted_by_user.
- Resume on process start: if a round is active in the committed prefs and the service is not recording, start it and log resume_on_process_start.

States (one service, D7):

State	Location updates	Wake lock	Fix log	Fixes to JS	Notification
idle	off	no	no	no	none (service stopped)
gps-on	on	no	no	while the Activity is resumed	"GPS on"
recording	on	yes	yes, round folder	while the Activity is resumed	"Recording round · started HH:MM"

- gps-on is entered when the shim's watchPosition is called with the Activity resumed, and left when the last watch is cleared or the Activity stops - unless recording, which keeps the service up regardless.
- recording is entered by startRecording(roundId) and left by stopRecording(roundId) with the same id (a different id is ignored and logged stop_ignored), by deleteRound(roundId), or by the cap.
- **Cap:** 8 h after startRecording, log auto_stop and stop. Longest logged round is 181 min (FT4, docs/HANDOFF-native-build.md Section 9) [measured]; the cap only bounds a round the web layer never closed.
- **Fix delivery to JS:** per fix, while the Activity is resumed, on the main thread:
webView.evaluateJavascript("window.__golfNativeFix(" + json + ")", null) with {lat, lon, acc, alt, altAcc, speed, heading, ts} where ts is the fix's own time (Location.time, the fix_ms column), absent values null. Nothing is queued while the Activity is stopped; gps.js's revive logic handles the resume (_bindVisibility, reviveGraceMs 3000 ms [measured]), and the shim's clearWatch/watchPosition pair must be idempotent for that restart.

4. The native track store

- **Root:** <filesDir>/rounds/<roundId>/ - roundId is the web record's id (r_<uuid>, filesystem-safe [measured]).
- **Files, same format as the bake-off** [measured]: fixes.csv (header wall_ms,fix_ms,elapsed_rt_ms,lat,lon,acc_m,alt_m,vacc_m,speed_mps,bearing_deg,provider,mock,moving,sample,event), events.csv (wall_ms,elapsed_rt_ms,kind,detail), meta.json (the bake-off's fields plus round_id, build = BUILD.id, recorder = "native-k"). Every cell measured, absent = empty, a fix with no time is not written (the event log records it).
- **Write posture, unchanged:** flush per row, fsync every 15 s and at close, cut-row repair on open, heartbeat every 5 s with the device state, the screen/Doze/battery-saver receivers, previous_exit.
- **Reading a round's track for the web layer** (GolfNative.readTrack): parse fixes.csv, skip the header and any cut row, skip rows with sample = 1, **de-duplicate on fix_ms** (first row wins; 2.3 Section 10), sort by fix_ms, and emit one compact point per row in the exact shape trackstore.js's compactFix writes [measured]:

[round(lat, 7), round(lon, 7), round(acc_m, 1), fix_ms], then speed_mps rounded to 2 dp appended only if present, then bearing_deg rounded to 0 dp appended only if speed_mps was present. acc_m absent → the row is skipped and counted (a fix with no accuracy cannot pass the gate anyway; count reported in stats). Result: a JSON array of arrays.

- **Importing a track** (GolfNative.importTrack(roundId, pointsJson)): points are compact arrays as above. Written as fixes.csv rows with wall_ms empty, fix_ms = p[3], lat, lon, acc_m, speed_mps (p[4] if present), bearing_deg (p[5] if present), provider = import, all other cells empty; one events.csv line imported with the point count; meta.json with recorder = "import". Appends if the folder exists (the web writeTrackChunk semantics: add, never replace [measured]). Returns the number written; 0 means nothing was stored.
- **Delete** (GolfNative.deleteTrack(roundId)): stops recording if that round is recording, removes the folder. Returns whether a folder existed (the web deleteTrack contract: *did I remove a track?* [measured]).
- **List / size / stats**: folder names; row count of fixes.csv (cheap line count minus header, cut row excluded); stats = {recording: bool, roundId, rows, lastFixMs, writeFailures, skippedNoAcc}.
- **Export of the logs for the PC tool** (GolfNative.exportLogs(roundId)): copies the folder to Download/golf-tracker/logs/<roundId>/ via MediaStore (the bake-off Exporter pattern). tools/track-coverage.py reads it as is [verify].

5. The bridge: window.GolfNative

All methods are @JavascriptInterface, synchronous, returning a String (JSON) or a primitive. Any failure returns a JSON {error: "<message>"} (for object results) or false/0 (for primitives) and logs an event; the web layer treats those exactly as it treats an unavailable IndexedDB today (degrade, never throw).

Method	Returns	Does
version()	"vNN"	BUILD.id baked into the APK
watch() / unwatch()	true	shim's watch count up/down → gps-on state
startRecording(roundId)	true if recording that id (idempotent)	D8, Section 3
stopRecording(roundId)	true if it stopped that id	same-id rule
recordingRoundId()	"r..." or ""	for the follow logic and the Data card
readTrack(roundId)	JSON array of compact points	Section 4
importTrack(roundId, pointsJson)	number written	Section 4
trackSize(roundId)	number	Section 4
trackedRoundIds()	JSON array of ids	folder names
deleteTrack(roundId)	true if a folder existed	Section 4
stats()	JSON {recording, roundId, rows, lastFixMs, writeFailures, skippedNoAcc}	Data card
saveExport(filename, json)	JSON {path} or {error}	writes to Download/golf-tracker/<filename> via MediaStore
exportLogs(roundId)	JSON {path} or {error}	Section 4

Native → JS, only while resumed: window.__golfNativeFix(fixJson).

6. Web-side changes, file by file (surgical; nothing else moves)

1. js/native/shim.js (**new**). When globalThis.GolfNative exists: define navigator.geolocation (the sim.js defineProperty pattern [measured]) with watchPosition(ok, err, opts) → registers ok, calls GolfNative.watch(), returns an id; clearWatch(id) → unregisters, calls GolfNative.unwatch(); getCurrentPosition resolves with the last fix if younger than 4 s else calls err with code: 2. Installs window.__golfNativeFix(f) that builds {coords: {latitude, longitude, accuracy, altitude, altitudeAccuracy, speed, heading}, timestamp: f.ts} and calls every registered ok. Exports nothing else.

2. `js/app.js`. (a) Before the GPS service can start, mirror the `?sim` block at line 346 [measured]: if (globalThis.GolfNative) await import('./native/shim.js'); when both `?sim=1` and the shell are present the sim wins (dev only). (b) Guard the service-worker registration at line 335: not in the shell. (c) `syncNativeRecorder(round)`: called from the same three places the track writer follows the round (inside the `ctx.gps.subscribe` handler at line 220 [measured]), plus once after the app state and active round are loaded at boot, plus on `visibilitychange` to visible. Rules: `round.status === 'in_progress' → GolfNative.startRecording(round.id)`; completed or abandoned `→ stopRecording(round.id)`; null round `→ nothing` (D8). (d) In the shell, `syncTrackWriter` creates no IndexedDB writer (denseTrack path skipped); `ctx.trackStats` (line 62) returns `GolfNative.stats()` mapped to the writer's {buffered, written, flushes, failures, inBuffer} shape so the Data card renders unchanged: `written = rows, failures = writeFailures, buffered = rows, flushes = 0, inBuffer = 0`. The decimated breadcrumb (`appendTrack, recordTrack`) is untouched. (e) In the shell, `startGps()` (line 132 [measured]) skips `wakeLock.acquire()` and `pocketLock.enable()`, and `stopGps()` their releases: the phone's power button is the lock and the recorder runs with the screen off (his word, 2026-09-16). `js/ui/lock.js` is not changed; it is simply never enabled in the shell.
3. `js/data/trackstore.js`. At the top of each public function, if `globalThis.GolfNative` exists, route to the bridge and return the same shape: `readTrack → JSON.parse(GolfNative.readTrack(id))` (on {error} or a parse failure return []); `writeTrackChunk(id, pts) → filter as today, then GolfNative.importTrack(id, JSON.stringify(pts))`; `trackSize, deleteTrack, trackedRoundIds` likewise; `pruneOrphanTracks` is unchanged (it composes the two above); `createTrackWriter` in the shell returns a writer whose push returns false, `flush/close` resolve, `stats()` maps `GolfNative.stats()` as in 2(d); `openTrackDb` is untouched and unused in the shell; `expandFix` untouched. **Every IndexedDB line stays as it is** - the Pages build must not change behaviour.
4. `js/data/persistence.js`. `checkPersistence` and `requestPersistence` return `PERSISTENT` when `GolfNative` exists (before touching `navigator.storage`). `persistenceLabel(PERSISTENT)` gets a second detail string for the shell: *"Rounds live in this app's private storage. Only clearing the app's data or uninstalling deletes them. Export after every round; the export is the only copy that survives that."* (the label function takes an optional { shell } flag; existing callers pass nothing and see today's text).
5. `js/data/store.js`. `downloadExport` and `shareExport`: in the shell, build the payload as today, call `GolfNative.saveExport(exportFilename(), JSON.stringify(payload, null, 2))`, and return {rounds, trackPoints, saved: path} (or shared: false, reason: 'failed' on {error}). Nothing else in the file changes; `buildExportWithTracks` already reads tracks through `readTrack`, so the export carries the native track without a line changing [measured].
6. `js/data/schema.js`. `newRound`'s device gains recorder: `globalThis.GolfNative ? 'native-k' : 'web'` (line 401 [measured]). Additive, optional; `schemaVersion` unchanged; `migrate()` unchanged; older records simply lack the key.
7. `js/ui/screen-settings.js`. In the shell only: a button *EXPORT RECORDER LOGS* for the active or last round `→ GolfNative.exportLogs(id) → toast the path`; the export buttons toast saved path instead of the share/download words. The persistence card passes { shell: true }. The pocket-lock settings (auto-lock and its timing) are hidden in the shell, since the lock never runs there.
8. `index.html`. No change unless the asset copy needs a relative path fix [verify].

7. What must not change

- `js/gps/gps.js, js/util/geo.js, js/round/track-analysis.js, js/round/round.js, js/analysis/*, sw.js`, the export formatVersion, schemaVersion, REVISION, the round record's existing keys, the IndexedDB code paths as run on Pages, android/bakeoff/*.
 - The pocket lock and the screen wake lock stay in the web build. In the shell both are off (his word, 2026-09-16, Section 10 item 2).
 - The suite's existing tests all pass as they are; `GpsService` tests run against the real `navigator.geolocation` exactly as now.
-

8. Moving the logged rounds across (his rounds, real data)

Procedure, on his phone, after the shell passes Section 9's gate:

1. In the PWA (Chrome): Settings → export (share to Drive or save). Note the toast's round and track-point counts.
2. Install the shell (`adb install`), grant the checklist.
3. In the shell: Settings → import → pick that file → merge.
4. Read back: rounds added, tracks restored, point counts - must equal the export's toast numbers.

Acceptance on this PC, before it goes near his phone (Opus runs it on the emulator with the real exports, docs/roundDownloads/*.json, gitignored, present on this PC [measured]): for every round in every export, after import, `readTrack(id).length` equals the export's `tracks[id].length`, and the first and last ts match. Known values to hit [measured this session]: export `golf-tracker-20260914-1807.json` has 6 rounds and 20,090 track points; round `r_18b4b0bb-701c-4bd8-83a5-a728617c2cde` has 8,579 points, first ts 1789420084966, last ts 1789427249443. Report every round's numbers with n.

Nothing in the shell rewrites a round record on import beyond what `migrate()` does today.

9. Tests and the gate

Browser suite (/test/), added, each proven to fail without its change:

1. The shim installs only when `GolfNative` exists; `watchPosition + __golfNativeFix` deliver a `GeolocationPosition`-shaped fix with `timestamp === fix.ts` to `GpsService._onFix` → `gps.last.ts` equals it.
2. `clearWatch` then `watchPosition` (the revive path) keeps delivering.
3. `trackstore.js` with a fake `GolfNative`: `readTrack` returns the parsed array; `writeTrackChunk` sends the filtered points and returns the native count; `deleteTrack` returns the bridge's boolean; `trackSize`, `trackedRoundIds` route; with `GolfNative` absent the `IndexedDB` path runs (the existing tests are that proof).
4. `persistence.js` returns `PERSISTENT` in the shell; the shell label text.
5. `newRound().device.recorder` is 'web' here and 'native-k' with a fake bridge.
6. Export in the shell calls `saveExport` with the same payload the download path builds (compare `format`, `formatVersion`, `rounds.length`, `trackPoints`).

Kotlin JVM unit tests (the bake-off pattern, `CoverageTest.kt` [measured]):

7. CSV → compact: rounding (7/7/1/2/0 dp), speed absent → 4 slots, bearing without speed → 4 slots, `acc_m` empty → skipped and counted, `sample=1` skipped, duplicate `fix_ms` → first wins, unsorted rows → sorted, a cut last row → skipped.
8. Import → CSV → read round-trips a fixture of compact points exactly (including a 6-slot point and a 4-slot point).
9. `stopRecording` with the wrong id is ignored; the cap fires at 8 h on a fake clock.

Emulator smoke (golf-bakeoff AVD, adb emu geo fix walk, the bake-off README recipe [measured]), n = 1 each, numbers in the report:

10. Start a round in the WebView; HOME; 60 s; recents → Close all; 60 s; reopen: the round is still `in_progress`, the notification never left, `readTrack` spans the closed interval with no gap > 2 s.
11. `kill -9` the process mid-round; 60 s; reopen: `resume_on_process_start` in `events.csv`, the round resumes, the gap is the restart backoff and nothing else.
12. Finish the round: recording stops, the notification clears, the export saved to `Download/golf-tracker/` carries the track with the same point count `readTrack` reports.
13. Section 8's import acceptance on the real exports.
14. `aapt dump permissions` shows no `INTERNET`.

The gate before it feeds a real round (his phone, his time, his word): the bar he set - 99% coverage and no gap over 20 s, on the Section 9 measure, across a round-length carry with the screen locked and a music app in use - measured by `tools/track-coverage.py` on the exported logs, with `golf-tracker`'s web build closed. Fable scores it and says PASS or FAIL in his chat. Until then the web build is the instrument.

10. What Matt decided (2026-09-16, his words: "go ahead with all 3")

1. REVISION 5 for the shell's first played round, with its REVISION_HISTORY line: **approved in advance**. Applied in the commit that makes the shell the instrument, after Section 9's gate - not in this build, because js/data/revision.js also ships to Pages and this week's web rounds stay rev 4.
 2. The in-app pocket lock and the screen wake lock are off in the shell; the power button is the lock (Section 6, items 2(e) and 7).
 3. If a K-alone carry fails the bar and a T-alone carry passes it, the recorder is swapped behind the Section 3 contract. A new item, priced first, when that evidence exists.
-

11. Order of work for Opus (one job, one commit per numbered step is fine)

1. android/tracker/ project skeleton, asset copy task, versionName from BUILD.id, no INTERNET, Setup screen, WebView with asset loader and file chooser, GolfNative with version() only. Emulator: the app loads and Settings shows the build.
2. Recorder + store lifted from the bake-off, renamed, Section 3 states, Section 4 files, the bridge complete. Kotlin tests 7-9.
3. Web side, Section 6, with browser tests 1-6. Suite green.
4. Emulator smoke 10-14; Section 8 acceptance on the real exports.
5. docs/handoff/REPORT_3.1.md + PDF: every number with n, the diff list, any departure from this spec and why, and the phone install recipe (adb install -r, grants, the checklist) as a device-first numbered list for Matt.

Return DONE <hash> or BLOCKED <question>. A spec line that cannot work as written is a BLOCKED with what was found, not a quiet variation.